

what about Data Units (PDU)

PDU stands for **Protocol Data Unit**. [↗](#)

It is the umbrella technical term that covers all the concepts we just discussed (Segment, Packet, Frame). A PDU is simply the complete block of information—including headers, trailers, and control data—that is transmitted between matching layers across a network. [↗](#)

Instead of coming up with a new name for every single layer, engineers use "PDU" as a universal template. [↗](#)

The OSI Model PDU Breakdown

As data travels down the 7 layers of the Open Systems Interconnection (OSI) model, the PDU changes its name based exactly on which layer is processing it: [↗](#)

OSI Layer	Layer Name	PDU Name	What it Contains
Layer 7	Application	Data / Message	Raw application information (e.g., HTTP payload, JPEG image)
Layer 6	Presentation	Data / Message	Encrypted or formatted application data
Layer 5	Session	Data / Message	Connection management data tokens
Layer 4	Transport	Segment (TCP) / Datagram (UDP)	Application data + Port Numbers [2]
Layer 3	Network	Packet	Segment + IP Addresses [2]
Layer 2	Data Link	Frame	Packet + MAC Addresses + Error Checkers [2]

Layer 1	Physical	Bits	The Frame converted into raw binary <code>0</code> s and <code>1</code> s (pulses) [2]
----------------	----------	-------------	--

PDU Peer-to-Peer Communication

The reason the concept of a PDU is so important is due to **Peer-to-Peer communication** across a network. [↗](#)

When your computer's Network Layer (Layer 3) creates a **Packet PDU**, it is designing that specific PDU to be read *only* by another Layer 3 device (like a Router or the destination computer's OS kernel). [↗](#)

- A Switch (Layer 2) cannot read a Layer 3 PDU wrapper; it only handles the Layer 2 Frame PDU wrapper.
- Each layer on the receiving computer peels off its corresponding PDU wrapper and passes the inner payload up to the next layer. [↗](#)

Summary Mnemonics

To easily remember the PDU order from Layer 7 down to Layer 1, network engineers use this classic phrase:

- **Dont** — **Data** (Layer 7, 6, 5)
- **Smoke** — **Segment** (Layer 4)
- **Pizza** — **Packet** (Layer 3)
- **Frozen** — **Frame** (Layer 2)
- **Burritos** — **Bits** (Layer 1) [↗](#)

If you want to dive deeper into how network protocol stacks handle these data objects, let me know:

- Do you want to see how the **Linux Kernel source code** represents a PDU using the `sk_buff` (**socket buffer**) structure?
- Would you like to look at **SDU (Service Data Unit)**, which is the companion concept to PDU?



What is a Protocol Data Unit (PDU)? | Definition from TechTarget

18 Feb 2025 — What is a protocol data unit (PDU)? What does a PDU contain?
How PDUs work PDUs for commonly used protocols



Protocol Data Unit (PDU) Meaning in Networking | OSI & TCP/IP Explained

27 Nov 2025 — Simple definition: A PDU is the specific form that data takes at each layer of a communication protocol. At one layer it may be ca...



Difference between PDU and MTU

19 Nov 2021 — A PDU - Protocol Data Unit is a term used to define your load (data) at different steps of the OSI model. Underneath the umbrella ...



Show all

what means of v

In the line you highlighted, the letter "**v**" does not actually appear as a standalone technical term or network variable. Instead, the letter "v" was a small formatting typo in the table—it was meant to be a **plus sign (+)** or an arrow to show composition.

Here is the exact meaning of what that specific Layer 3 row represents:

The Meaning of Layer 3: Packet Composition

That line describes how a Network Layer PDU (Protocol Data Unit) is built during network encapsulation: 🔗

Breaking Down the Components:

1. **Layer 3 (Network Layer):** The conceptual layer of the network stack responsible for routing data across different networks globally.
2. **Packet:** The name of the data unit at this specific layer.
3. **Segment + IP Addresses:** This explicitly defines what a Packet is made of. The operating system kernel takes the **Segment** (which contains the raw data and port numbers from Layer 4) and wraps an **IP Header** (containing the source and destination IP addresses) directly around it. [🔗](#)

Common Network Confusions with "V"

If you are seeing the letter **V** used elsewhere in network programming or architecture discussions around Layer 3 or Layer 2, it typically refers to one of these three common terms:

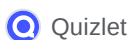
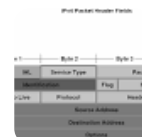
- **VLAN (Virtual Local Area Network):** A method to split a single physical switch into multiple isolated virtual networks at Layer 2.
- **IPv4 / IPv6:** The **V** stands for **Version** (Internet Protocol Version 4 or Version 6).
- **V-Switch (Virtual Switch):** A software-defined network switch running inside a hypervisor (like VMware or KVM) to connect virtual machines. [🔗](#)

Let me know if you saw the letter "**v**" in a specific code structure, diagram, or error message, and I can tell you exactly what it means in that context.



OSI Network Layer OSI Layer 3

Second, the Network layer must provide encapsulation. During the encapsulation process, Layer 3 receives the Layer 4 PDU and adds ...



CCNA 200-301 Key Vocabulary Terms. Flashcards

Protocol Data Unit. A generic term referring to the header defined by some layer of a networking model, and the data encapsulated ...





what means of Port Numbers [2]

A **Port Number** is a 16-bit unsigned integer (a value from 0 to 65535) used by the operating system to route network data to a **specific application or process** on your computer.

While an IP address gets a data packet to the correct physical computer, the port number ensures that the data reaches the exact software program waiting for it.

The Analogy: Apartment Building vs. Room Number

- **IP Address:** The street address of a large apartment building. It gets the mail truck to the right property.
- **Port Number:** The specific apartment or room number inside that building. Without it, the mail carrier doesn't know which resident should receive the letter.

How Port Numbers Work in the PDU Stack

Port numbers belong to **Layer 4 (the Transport Layer)** of the network stack, wrapped inside a **Segment** (TCP) or **Datagram** (UDP).

Every network connection uses a pair of ports:

1. **Source Port:** A random port chosen automatically by your operating system's kernel (usually a high number like 51020) to track your specific browser tab or application window.
2. **Destination Port:** The specific port number of the service you are trying to reach on the remote server.

The Three Port Number Categories

The Internet Assigned Numbers Authority (IANA) divides the 65,536 available ports into three distinct ranges:

1. Well-Known Ports (0 – 1023)

These are reserved for standard, system-level core internet services. On Linux and macOS, an application must have administrator (`root`) privileges to bind to these ports.

- **Port 80:** HTTP (Unencrypted Web Browsing)
- **Port 443:** HTTPS (Secure/Encrypted Web Browsing)
- **Port 22:** SSH (Secure Remote Server Access)
- **Port 53:** DNS (Domain Name System resolution)

2. Registered Ports (1024 – 49151)

These are used by third-party vendor applications and databases. They do not require admin privileges to open.

- **Port 3306:** MySQL Database
- **Port 5432:** PostgreSQL Database
- **Port 8080:** Alternative HTTP port commonly used for local software development test servers.

3. Dynamic / Private Ports (49152 – 65535)

Also known as **Ephemeral Ports**. Your operating system dynamically assigns these to client applications (like your Chrome browser) as temporary source ports when initiating an outbound connection. They are automatically recycled and freed when you close the connection or browser tab.

Code Connection: Sockets

In network programming (like C++ or Python socket programming), you combine an IP address and a Port Number together to form a **Socket Endpoint**.

```
text
```

```
192.168.1.50    :    8080
[ IP Address ]  [ Port Number ]
```

\ _____ /
Socket Endpoint

Use code with caution.

If you want to see how this works on your machine, let me know:

- Do you want to see how to use **terminal commands** like `netstat` or `ss` to view all active port connections on your PC?
- Would you like to see a **C++ socket code snippet** showing how an application binds to a specific port?